Date __/__/__
Page _____

Lab-2

1. C program to simulate cpu scheduling algorithm FCFS.

Code

```
#include <stdio.h>
int main(){
    int n;
    printf("enter number of processes: ");
    scanf("%d", &n);
    int pid[n], at[n], bt[n];
    int ct[n], tat[n], wt[n], rt[n];
    int start[n];
    for(int i=0; i<n; i++){
        pid[i]= i;
        printf("enter AT & BT for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    for(i=0; i<n-1; i++){
        for(j=0; j<n; j++){
            if(at[pid[i]] > at[pid[j]]){
                int temp = pid[i];
                pid[i] = pid[j];
                pid[j] = temp;
            }
        }
    }
    int time = 0;
    for(i=0; i<n; i++){
        int p = pid[i];
        if(time < at[p]) time = at[p];
        start[p] = time;
        rt[p] = start[p] - at[p];
        time = time + bt[p];
```

```
        ct[p] = time;
        tat[p] = ct[p] - at[p];
        wt[p] = tat[p] - bt[p];
    }
    printf("\nProcess\t AT \t BT \t CT \t TAT \t WT \t RT\n");
    for(i=0; i<n; i++){
        printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
    return 0;
}
```

Output:

```
Enter number of processes: 3
Enter AT and BT for P1: 1 4
Enter AT and BT for P2: 0 3
Enter AT and BT for P3: 2 6
```

Process	AT	BT	CT	TAT	WT	RT
P1	1	4	7	6	2	2
P2	0	3	3	3	0	0
P3	2	6	13	11	5	5

② C program to simulate CPU scheduling algorithm ~~for~~ SJF.

Code:

```
#include<stdio.h>
int main(){
    int n, i, choice;
    printf("enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n];
    int ct[n], tat[n], wt[n], rt[n];
    for(i=0; i<n; i++){
        printf("Enter AT and BT for P%d: ", i+1);
        scanf("%d%d", &at[i], &bt[i]);
    }
    printf("\n1. SJF Non Preemptive");
    printf("\n2. SJF Preemptive");
    printf("\nEnter choice: ");
    scanf("%d", &choice);
    if(choice==1){
        int time=0, completed=0;
        int visited[10]={0};
        while(completed<n){
            int idx=-1, min=9999;
            for(i=0; i<n; i++){
                if(at[i]<=time && visited[i]==0 &&
                        bt[i]<min){
                    min=bt[i];
                    idx=i;
                }
            }
            if(idx!=-1){
                rt[idx]=time-at[idx];
                time+=bt[idx];
```

```
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            visited[idx] = 1;
            completed++;
        }
        else
            time++;
    }
}
else if (choice == 2) {
    int time = 0, completed = 0;
    int rbt[10];
    int first[10] = {0};
    for (i = 0; i < n; i++)
        rbt[i] = bt[i];
    while (completed < n) {
        int idx = -1, min = 9999;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && rbt[i] > 0 &&
                rbt[i] < min) {
                min = rbt[i];
                idx = i;
            }
        }
        if (idx != -1) {
            if (first[idx] == 0) {
                rt[idx] = time - at[idx];
                first[idx] = 1;
            }
            rbt[idx]--;
            time++;
            if (rbt[idx] == 0) {
                ct[idx] = time;
                tat[idx] = ct[idx] - at[idx];
                wt[idx] = tat[idx] - bt[idx];
                completed++;
            }
        }
        else
            time++;
    }
    printf("\nProcess \t AT \t BT \t CT \t TAT \t WT \t RT\n");
    for (i = 0; i < n; i++) {
        printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
    return 0;
}
```

Output:

```
Enter number of processes: 4
Enter AT and BT for P1: 1 4
Enter AT and BT for P2: 2 2
Enter AT and BT for P3: 0 3
Enter AT and BT for P4: 3 5
1. SJF Non Preemptive
2. SJF Preemptive
Enter choice: 1
```

Process	AT	BT	CT	TAT	WT	RT
P1	1	4	9	8	4	4
P2	2	2	5	3	1	1
P3	0	3	3	3	0	0
P4	3	5	14	11	6	6

Enter number of processes: 3
Enter AT and BT for P1 : 0 8
Enter AT and BT for P2 : 1 4
Enter AT and BT for P3 : 2 2
1. SJF Non Preemptive
2. SJF Preemptive
Enter choice 2

Process	AT	BT	CT	TAT	WT	RT
P1	0	8	14	14	6	0
P2	1	4	7	6	2	0
P3	2	2	4	2	0	0

FCFS (vs) SJF

SJF is more optimal than FCFS.

* In FCFS, processes are executed in order of arrival (First come First serve)
* Once process starts, it runs until completion.
* It causes delay for all small processes when a long process arrives first.
* It gives higher average waiting time.

* In SJF, CPU selects process with smallest burst time.
* shortest process runs first.
* It produces less average waiting time.

LAB-3

Write C program to simulate CPU scheduling algorithm to find TAT & WT for Priority (Preemptive & non-preemptive)

Code:

```
#include<stdio.h>
int main(){
    int n, i, time=0, completed=0, choice;
    int at[10], bt[10], pr[10], rt[10];
    int ct[10], tat[10], wt[10], res[10];
    int start[10] = {0};
    int visited[10] = {0};
    printf("enter number of processes: ");
    scanf("%d", &n);
    for(i=0; i<n; i++){
        printf("Enter AT & BT & Priority for P%d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
        start[i] = -1;
    }
    printf("\n1. Priority Non-Preemptive\n");
    printf("2. Priority, Preemptive\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    float avg_tat, avg_wt, avg_rt;
    printf("\nGantt Chart:\n");
    if(choice == 1){
        while(completed < n){
            int idx = -1;
            int high = 999;
            for(i=0; i<n; i++){
                if(at[i] <= time && visited[i] == 0){
                    if(pr[i] < high){
```

```
            high = px[i];
            idx = i;
        }
    }
}
if (idx != -1) {
    start[idx] = time;
    res[idx] = start[idx] - at[idx];
    printf("P%d ", idx+1);
    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    visited[idx] = 1;
    completed++;
}
else
    time++;
}
else {
while (completed < n) {
    int idx = -1;
    int high = 999;
    for (i=0; i<n; i++) {
        if (at[i] <= time && x[i] > 0) {
            if (px[i] < high) {
                high = px[i];
                idx = i;
            }
        }
    }
if (idx != -1) {
    if (start[idx] == -1) {
        start[idx] = time;
        res[idx] = start[idx] - at[idx];
    }
    printf("P%d ", idx+1);
    x[idx] -= q;
    time++;
    if (x[idx] == 0) {
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        completed++;
    }
}
else
    time++;
}
printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
for (i=0; i<n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1, at[i], bt[i], px[i], ct[i], tat[i], wt[i], res[i]);
    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += res[i];
}
avg_tat /= n;
avg_wt /= n;
avg_rt /= n;
printf("\nAverage TAT = %.2f", avg_tat);
printf("\nAverage WT = %.2f", avg_wt);
printf("\nAverage RT = %.2f", avg_rt);
}
```

O/P:

Enter number of processes: 3
Enter AT BT Priority for P1: 1 4 3
Enter AT BT Priority for P2: 0 1 2
Enter AT BT Priority for P3: 2 6 1
1. Priority Non-preemptive
2. Priority Preemptive
Enter choice: 1
Gantt chart:
|P2|P1|P1|P3|

PID	AT	BT	PR	CT	TAT	WT	RT
P1	1	4	3	5	4	0	0
P2	0	1	2	1	1	0	0
P3	2	6	1	11	9	3	3

Average TAT = 4.67
Average WT = 1.00
Average RT = 1.00

Enter choice: 2
Gantt chart:
|P2|P1|P3|P3|P3|P3|P3|P3|P1|P1|P1|P1|

PID	AT	BT	PR	CT	TAT	WT	RT
P1	1	4	3	11	10	6	0
P2	0	1	2	1	1	0	0
P3	2	6	1	8	6	0	0

Average TAT = 5.67
Average WT = 2.00
Average RT = 0.00

4/3/26

Date / /
Page

Lab-4

Round Robin

```
#include<stdio.h>
int main(){
    int n, i, time=0, tq, remain;
    int at[10], bt[10], rt[10], ct[10], tat[10], wt[10],
        response[10];
    int first_exec[10], gantt_p[100], gantt_t[100];
    int g_index=0;
    float total_wt=0, total_tat=0, total_rt=0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    remain=n;
    for(i=0; i<n; i++){
        printf("Enter arrival & burst time for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i]=bt[i];
        first_exec[i]=-1;
    }
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    while(remain!=0){
        int done=1;
        for(i=0; i<n; i++){
            if(rt[i]>0 && at[i]<=time){
                done=0;
                if(first_exec[i]==-1){
                    first_exec[i]=time;
                    response[i]=time-at[i];
                }
                gantt_p[g_index]=i;
                gantt_t[g_index]=time;
                g_index++;
```

```
if (bt[i] > tq){
    time += tq;
    bt[i] -= tq;
} else {
    time += bt[i];
    ct[i] = time;
    bt[i] = 0;
    remain--;
}
}
if (done){
    time++;
}
}
gantt_t[g_index] = time;
for(i=0; i<n; i++){
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
    total_rt += response[i];
}
printf("Gantt chart:\n");
printf("|");
for(i=0; i<g_index; i++){
    printf(" P%d |", gantt_pid[i]+1);
}
for(i=0; i<g_index; i++){
    printf(" %d ", gantt_t[i]);
}
printf("\n\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(i=0; i<n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], response[i]);
}
printf("\nAverage TAT = %.2f", total_tat/n);
printf("\nAverage WT = %.2f", total_wt/n);
printf("\nAverage RT = %.2f", total_rt/n);
return 0;
}
```

Output: enter number of processes: 3
enter AT & BT for P1: 0 5
enter AT & BT for P2: 1 8
enter AT & BT for P3: 2 3
enter time quantum: 3

Gantt chart: | P1 | P2 | P3 | P1 | P2 | P2 |
0 3 6 9 11 14 16

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	11	11	6	0
P2	1	8	16	15	7	2
P3	2	3	9	7	4	4

Avg TAT = 11.00
Avg WT = 5.67
Avg RT = 2.00

enter time quantum: 4

Gantt chart: | P1 | P2 | P3 | P1 | P2 |
0 4 8 11 12 16

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	12	12	7	0
P2	1	8	16	15	7	3
P3	2	3	11	9	6	6

Avg TAT = 12.00
Avg WT = 6.67
Avg RT = 3.00

→ As we increase time quantum response time will increase, for more efficiency, time quantum should be

Multilevel

```
#include<stdio.h>
#include<string.h>
#define MAX 20
int main(){
    int n,i,j,time=0,comp=0,idx=0,cur=-1;
    pid[MAX], at[MAX], bt[MAX], rt[MAX], ct[MAX],
    tat[MAX], wt[MAX], rtm[MAX], fe[MAX];
    char type[MAX][10];
    int sq[MAX], uq[MAX], sf=0, sr=-1, uf=0, ur=-1;
    int gp[200], gt[200], g=0;
    printf("Enter n: "); scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter process, at & bt & type: ",i+1);
        scanf("%d%d%d%s",&pid[i],&at[i],&bt[i],type[i]);
        rt[i]=bt[i]; fe[i]=-1;
    }
    for(i=0;i<n-1;i++){
        for(j=i+1;j<n;j++){
            if(at[i]>at[j]){
                int t;
                t=at[i]; at[i]=at[j]; at[j]=t;
                t=bt[i]; bt[i]=bt[j]; bt[j]=t;
                t=rt[i]; rt[i]=rt[j]; rt[j]=t;
                t=pid[i]; pid[i]=pid[j]; pid[j]=t;
                char tmp[10]; strcpy(tmp,type[i]);
                strcpy(type[i],type[j]);
                strcpy(type[j],tmp);
            }
        }
    }
    while(comp<n){
        while(idx<n && at[idx]<=time){
            if(strcmp(type[idx],"system")==0)
                sq[++sr]=idx;
            else
                uq[++ur]=idx;
            idx++;
        }
        if(cur!=-1 && strcmp(type[cur],"user")==0 && sf<=sr)
            uq[++ur]=cur;
            cur=-1;
        }
        if(cur==-1){
            if(sf<=sr) cur=sq[sf++];
            else if(uf<=ur) cur=uq[uf++];
            else{
                time++;
                continue;
            }
        }
        if(fe[cur]==-1){
            fe[cur]=time;
            rtm[cur]=time-at[cur];
        }
        gp[g]=cur;
        gt[g++]=time;
        rt[cur]--;
        time++;
        if(rt[cur]==0){
            ct[cur]=time;
            printf("\nProcess P%d completed at time%d\n",
                    pid[cur],time);
            comp++;
            cur=-1;
        }
    }
    gt[g]=time;
```

```
float awt=0, atat=0, art=0;
printf("\nP AT BT CT TAT WT RT\n");
for(i=0; i<n; i++){
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
    awt+=wt[i]; atat+=tat[i]; art+=rt[i];
    printf("P%d %d %d %d %d %d %d\n",
        pid[i], at[i], bt[i], ct[i], tat[i], wt[i],
                rt[i]);
}
printf("\nAvg TAT=%.2f\n Avg WT=%.2f\n
        Avg RT=%.2f\n", atat/n, awt/n, art/n);
printf("\nGantt:\n");
for(i=0; i<g; i++) printf("P%d|", pid[gp[i]]);
printf("\n%d", gt[0]);
for(i=1; i<=g; i++) printf(" %d", gt[i]);
return 0;
}
```

Output: enter n: 3

enter process, at, bt & type: 105 user
enter process, at, bt & type: 213 system
enter process, at, bt & type: 322 user
process P2 completed at time 4
process P1 completed at time 8
process P3 completed at time 10

P	AT	BT	CT	TAT	WT	RT
P1	0	5	8	8	3	0
P2	1	3	4	3	0	0
P3	2	2	10	8	6	6

Avg TAT = 6.33, Avg WT = 3.00, Avg RT = 2.00

Gantt:

|P1|P2|P2|P2|P1|P1|P1|P1|P3|P3|

0 1 4 8 10

LAB-5

C program to simulate Real time CPU scheduling algorithms
* Rate-Monotonic * Earliest deadline First * Proportional scheduling

Code:

```
#include <stdio.h>
#include <math.h>
int main(){
    int n, i, j, temp, time;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int b[n], d[n], p[n], id[n];
    for(i=0; i<n; i++){
        printf("\nProcess %d:\n", i);
        id[i]=i;
        printf("Burst time: "); scanf("%d", &b[i]);
        printf("Deadline: "); scanf("%d", &d[i]);
        printf("Time period: "); scanf("%d", &p[i]);
    }
    float util=0;
    for(i=0; i<n; i++) util+=(float)b[i]/p[i];
    // EDF
    for(i=0; i<n-1; i++)
        for(j=i+1; j<n; j++)
            if(d[i]>d[j]){
                temp=d[i]; d[i]=d[j]; d[j]=temp;
                temp=b[i]; b[i]=b[j]; b[j]=temp;
                temp=p[i]; p[i]=p[j]; p[j]=temp;
            }
    printf("\nEDF Scheduling\nCPU utilization: %f\n",
                util);
    printf("ID CT WT TAT\n");
```

```
time = 0;
for (i=0; i<n; i++){
    time += b[i];
    printf("%d %d %d %d\n", id[i], time,
                time - b[i], time);
}
// RMS
for(i=0; i<n-1; i++)
    for (j=i+1; j<n; j++)
        if (p[i] > p[j]){
            temp = p[i]; p[i] = p[j]; p[j] = temp;
            temp = b[i]; b[i] = b[j]; b[j] = temp;
            temp = id[i]; id[i] = id[j]; id[j] = temp;
        }
float bound = n*(pow(2, 1.0/n) - 1);
printf("\n RMS Scheduling \n CPU Utilization : %.2f\n
                RM Bound : %.4f\n", util,
                bound);
printf("ID CT WT TAT\n");
time = 0;
for(i=0; i<n; i++){
    time += b[i];
    printf("%d %d %d %d\n", id[i], time,
                time - b[i], time);
}
// Proportional Scheduling
printf("\n = = = = Proportional Scheduling = = = =\n");
printf("ID Burst share\n");
int total = 0;
for (i=0; i<n; i++) total += b[i];
for(i=0; i<n; i++){
    float share = (float) b[i] / total;
    printf("%d %d %.2f\n", id[i], b[i], share);
}
```

Output:

```
enter number of processes : 3
process 0 : Burst time : 3
            Deadline : 7
            Period : 5
process 1 : Burst time : 2
            Deadline : 4
            Period : 5
process 2 : Burst time : 2
            Deadline : 8
            Period : 10
EDF Scheduling
CPU Utilization : 0.75
ID  CT  WT  TAT
1   2   0   2
0   5   2   5
2   7   5   7
RMS Scheduling
CPU Utilization : 0.75
RMS bound : 0.7798
ID  CT  WT  TAT
0   3   0   3
2   5   3   5
1   7   5   7
= = = = Proportional Scheduling = = = =
ID  Burst  Share
0   3      0.43
2   2      0.29
1   2      0.29
```

24-4-26 Lab6

Bounded buffer

~~Tracing~~ O/p: ~~Producer~~

Tracing: (mutex(0) - locked, (1) - free)

empty = 5, full = 0, mutex = 0, in = 0, out = 0.

Sno	operation	Buffer state	in	out	empty	full	mutex
1.	Producer: 0	[0,_,_,_,_]	1	0	4	1	1
	Consumer: 0	[_,_,_,_,_]	1	1	5	0	1
2.	Producer: 1	[_,1,_,_,_]	2	1	4	1	1
	Consumer: 1	[_,_,_,_,_]	2	2	5	0	1
3.	Producer: 2	[_,_,2,_,_]	3	2	4	1	1
	Producer: 3	[_,_,2,3,_]	4	2	3	2	1
4.	Consumer: 2	[_,_,_,3,_]	4	3	4	1	1

Dining philosopher:

Step-1: All thinking

Step-2: P0 picked up left fork 0

P0 picked up right fork 1

P0 is eating

P1 picked up right fork 2

P1 picked up left fork 1

P1 is eating

P0 puts down forks 0 and 1

P3 picked up right fork 4

P3 picked up left fork 3

P3 is eating

Code: Bounded buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define MAX_ITEMS 15
int buffer[BUFFER_SIZE];
int in=0, out=0;
sem_t empty, full;
pthread_mutex_t mutex;
void *producer(void *arg){
  for(int i=0; i<MAX_ITEMS; i++){
    sem_wait(&empty);
    pthread_mutex_lock(&mutex);
    buffer[in] = i;
    printf("Produced: %d at buffer[%d]\n", i, in);
    in = (in+1)%BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
    sem_post(&full);
    usleep(100000);
  }
  return NULL;
}
void *consumer(void *arg){
  for(int i=0; i<MAX_ITEMS; i++){
    sem_wait(&full);
    pthread_mutex_lock(&mutex);
    int item = buffer[out];
    printf("Consumed %d from buffer[%d]\n", item,
    out = (out+1)%BUFFER_SIZE;
    pthread_mutex_unlock(&mutex);
    sem_post(&empty);
```

```
        usleep(1500000);
    }
    return NULL;
}
int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

o/p:

```
Produced : 0 at buffer[0]
Consumed : 0 from buffer[0]
Produced : 1 at buffer[1]
Consumed : 1 from buffer[1]
Produced : 2 at buffer[2]
Consumed : 2 from buffer[2]
Produced : 3 at buffer[3]
Produced : 4 at buffer[4]
Consumed : 3 from buffer[3]
Produced : 5 from buffer[5]
.
.
.
Consumed : 14 from buffer[4]
```

Code : Dining Philosophers :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t forks[N];
void *philosopher(void *num){
    int id = *(int*)num;
    for(int i=0; i<1; i++){
        printf("Philosophier %d is thinking.\n", id);
        sleep(1);
        if(id%2==0){
            sem_wait(&forks[id]);
            sleep(1);
            printf("Philosopher %d picked up left
                    fork %d.\n", id, id);
            sem_wait(&forks[(id+1)%N]);
            printf("Philosopher %d picked up right
                    fork %d.\n", id, (id+1)%N);
        }else{
            sem_wait(&forks[(id+1)%N]);
            printf("Philosopher %d picked up right
                    fork %d.\n", id, (id+1)%N);
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left
                    fork %d.\n", id, id);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
        sem_post(&forks[id]);
        sem_post(&forks[(id+1)%N]);
        printf("Philosopher %d put down forks %d
```

Date / Page

06.05.26

```
        and %d.\n", id, id, (id+1)%N);
    }
    pthread_exit(NULL);
}
int main(){
    pthread_t ph[N];
    int ids[N];
    for(int i=0; i<N; i++){
        sem_init(&forks[i], 0, 1);
    }
    for (int i=0; i<N; i++){
        ids[i]=i;
        pthread_create(&ph[i], NULL, philosopher, &ids[i]);
    }
    for (int i=0; i<N; i++){
        pthread_join(ph[i], NULL);
    }
    return 0;
}
```

O/p:

```
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 4 is thinking
Philosopher 0 picked up left fork 0
Philosopher 1 picked up right fork 2
Philosopher 4 picked up left fork 4
Philosopher 0 picked up right fork 1
Philosopher 0 is eating.
Philosopher 0 put down forks 0 and 1
```

LAB-7

Bankers Algorithm for purpose of deadlock Avoidance

```
#include<stdio.h>
int main(){
    int i, j, k, n, m;
    printf("enter no. of processes and resources:\n");
    scanf("%d %d", &n, &m);
    int alloc[n][m], max[n][m], need[n][m], avail[m];
    printf("enter allocation matrix\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max Matrix:\n");
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            scanf("%d", &max[i][j]);
    printf("enter Available resources:\n");
    for(i=0; i<m; i++)
        scanf("%d", &avail[i]);
    for(i=0; i<n; i++)
        for(j=0; j<m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
//---------- Resource request ----------
    int p;
    int request[m];
    printf("enter process number requesting resources");
    scanf("%d", &p);
    printf("enter request vector:\n");
    for(i=0; i<m; i++)
        scanf("%d", &request[i]);
    for(i=0; i<m; i++)
        if(request[i] > need[p][i]){
            printf("Error: Process exceeded max claim");
            return 0;
```

```
for(i=0;i<n;i++){
  if(request[i]>avail[i]){
    printf("Resources not available.Process must wait\n");
    return 0;
  }
}
for(i=0;i<m;i++){
  avail[i]-=request[i];
  alloc[p][i]+=request[i];
  need[p][i]-=request[i];
}
// ---- Safety algorithm ----
int finish[n], safe[n], work[m];
for(i=0;i<m;i++) work[i]=avail[i];
for(i=0;i<n;i++) finish[i]=0;
int count=0;
while(count<n){
  int found=0;
  for(i=0;i<n;i++){
    if(finish[i]==0){
      for(j=0;j<m;j++){
        if(need[i][j]>work[j]) break;
      }
      if(j==m){
        for(k=0;k<m;k++)
          work[k]+=alloc[i][k];
        safe[count++]=i;
        finish[i]=1;
        found=1;
      }
    }
  }
  if(found==0) break;
}
if(count==n){
  printf("Request can be granted.\n");
  printf("System is in safe state.\n");
  printf("Safe sequence: ");
  for(i=0;i<n;i++)
    printf("P%d ", safe[i]);
  printf("\n");
}else{
  for(i=0;i<m;i++){
    avail[i]+=request[i];
    alloc[p][i]-=request[i];
    need[p][i]+=request[i];
  }
  printf("Request can't be granted.\n");
  printf("System would become unsafe.\n");
}
return 0;
}
```

O/P: enter number of processes and resources: 5 3
enter allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
enter Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
enter Available resources:
3 3 2
enter process number requesting resources:
1
enter request vector:
1 0 2
Request can be granted.
System is in safe state
Safe sequence: P1 P3 P4 P0 P2

Deadlock detection

```
#include<stdio.h>
int main(){
  int n,m,i,j,k;
  printf(" enter numbers of processes and resources");
  scanf("%d %d",&n,&m);
  int alloc[n][m], req[n][m], avail[m], finish[n];
  printf("Enter allocation matrix\n");
  for(i=0;i<n;i++)
    for(j=0;j<m;j++)
      scanf("%d",&alloc[i][j]);
  printf("enter request matrix:\n");
  for(i=0;i<n;i++)
    for(j=0;j<m;j++)
      scanf("%d",&req[i][j]);
  printf("enter Available resources\n");
  for(i=0;i<m;i++)
    scanf("%d",&avail[i]);
  for(i=0;i<n;i++)
    finish[i]=0;
  for(i=0;i<n;i++){
    if(!finish[i]){
      for(j=0;j<m;j++)
        if(req[i][j]>avail[j])
          break;
      if(j==m){
        for(k=0;k<m;k++)
          avail[k]+=alloc[i][k];
        finish[i]=1;
      }
    }
  }
  for(i=0;i<n;i++){
    if(!finish[i])
      printf("Deadlock at P%d\n",i);
  }
  return 0;
}
```

Op:

```
enter number of processes and resources: 5 3
enter allocation matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
enter request matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
enter Available resources:
0 0 0
Deadlock at P1.
```

8/5/26

15-5-26

Lab-8

Given 6 memory partitions of 300KB, 600KB, 350KB, 200KB, 750KB, 125KB (in order). How would best fit, worst fit, first fit algorithms place processes of size 115KB, 500KB, 358KB, 200KB, 375KB (in order)

First fit:

375x

Best fit:

375x

Worst fit:

358, 375x

C program to simulate following contiguous memory allocation techniques:

a) Worst fit b) Best fit c) First fit

Code

```
#include<stdio.h>
int main(){
    int np, nb, i, j, b[20], p[20], allocation[20];
    printf("enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("enter sizes of %d memory blocks:\n", nb);
    for(i=0; i<nb; i++) scanf("%d", &b[i]);
    printf("enter number of processes: ");
    scanf("%d", &np);
    printf("enter sizes of %d processes:\n", np);
    for(i=0; i<np; i++) scanf("%d", &p[i]);

    int temp[20];
    for(i=0; i<nb; i++) temp[i] = b[i];
    for(i=0; i<np; i++) allocation[i] = -1;
    for(i=0; i<np; i++){
        for(j=0; j<nb; j++){
            if(temp[j] >= p[i]){
                allocation[i] = j+1;
                temp[j] = -1;
                break;
            }
        }
    }
    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for(i=0; i<np; i++){
        if(allocation[i] != -1)
            printf("%d\t\t%d\t\t%d\n", i+1, p[i], allocation[i]);
        else
            printf("%d\t\t%d\t\tNot Allocated\n", i+1, p[i]);
    }
    for(i=0; i<nb; i++) temp[i] = b[i];
    for(i=0; i<np; i++) allocation[i] = -1;
    for(i=0; i<np; i++){
        int bestIdx = -1;
        for(j=0; j<nb; j++){
            if(temp[j] >= p[i]){
                if(bestIdx == -1 || temp[j] < temp[bestIdx])
                    bestIdx = j;
            }
        }
        if(bestIdx != -1){
            allocation[i] = bestIdx + 1;
            temp[bestIdx] = -1;
        }
    }
    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for(i=0; i<np; i++){
        if(allocation[i] != -1)
```

```
else
    printf("%d\t\t%d\t\tNot Allocated\n", i+1, p[i]);
}
for(i=0; i<nb; i++) temp[i] = b[i];
for(i=0; i<np; i++) allocation[i] = -1;
for(i=0; i<np; i++) {
    int worstIdx = -1;
    for(j=0; j<nb; j++) {
        if(temp[j] >= p[i]) {
            if(worstIdx == -1 || temp[j] > temp[worstIdx])
                worstIdx = j;
        }
    }
    if(worstIdx != -1) {
        allocation[i] = worstIdx + 1;
        temp[worstIdx] -= p[i];
    }
}
printf("\n--- Worst Fit ---\n");
printf("Process No\tProcess Size\tBlock No\n");
for(i=0; i<np; i++) {
    if(allocation[i] != -1)
        printf("%d\t\t%d\t\t%d\n", i+1, p[i], allocation[i]);
    else
        printf("%d\t\t%d\t\tNot Allocated\n", i+1, p[i]);
}
return 0;
}
```

O/p:

```
Enter no of memory blocks: 5
enter sizes of 5 memory blocks:
100 500 200 300 600
enter number of processes: 4
enter sizes of 4 processes:
212 417 112 426
--- First Fit ---
Process No      Process Size    Block No
1               212             2
2               417             3
3               112             3
4               426             Not Allocated
--- Best Fit ---
Process No      Process Size    Block No
1               212             4
2               417             2
3               112             3
4               426             5
--- Worst Fit ---
Process No      Process Size    Block No
1               212             5
2               417             2
3               112             4
4               426             Not Allocated
```

LAB-9

Write C program to simulate the following page replacement algorithms:

Code:

```
#include <stdio.h>
int frames[10];
int search(int key, int frameCount){
    for(int i=0; i<frameCount; i++){
        if(frames[i]==key) return 1;
    }
    return 0;
}

void FIFO(int pages[], int n, int frameCount){
    int pageFaults=0, index=0;
    for(int i=0; i<frameCount; i++) frames[i]=-1;
    printf("\n---- FIFO ----\n");
    for(int i=0; i<n; i++){
        if(!search(pages[i], frameCount)){
            frames[index]=pages[i];
            index=(index+1)%frameCount;
            pageFaults++;
        }
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<frameCount; j++){
            if(frames[j]!=-1) printf("%d ", frames[j]);
            else printf("- ");
        }
        printf("\n");
    }
    printf("Total page faults = %d\n", pageFaults);
}

void LRU(int pages[], int n, int frameCount){
    int pageFaults=0, time[10], counter=0;
    for(int i=0; i<frameCount; i++){
        frames[i]=-1;
        time[i]=0;
    }
    printf("\n---- LRU ----\n");
    for(int i=0; i<n; i++){
        int found=0;
        for(int j=0; j<frameCount; j++){
            if(frames[j]==pages[i]){
                counter++;
                time[j]=counter;
                found=1;
                break;
            }
        }
        if(!found){
            int pos=0;
            for(int j=1; j<frameCount; j++){
                if(time[j]<time[pos]) pos=j;
            }
            frames[pos]=pages[i];
            counter++;
            time[pos]=counter;
            pageFaults++;
        }
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<frameCount; j++){
            if(frames[j]!=-1) printf("%d ", frames[j]);
            else printf("- ");
        }
        printf("\n");
    }
    printf("Total Page faults = %d\n", pageFaults);
}
```

```
void optimal(int pages[], int n, int frameCount){
    int pageFaults = 0;
    for(int i=0; i<frameCount; i++) frames[i] = -1;
    printf("\n--- Optimal ---\n");
    for(int i=0; i<n; i++){
        if(!search(pages[i], frameCount)){
            printf("Page %d -> ", pages[i]);
            for(int j=0; j<frameCount; j++){
                if(frames[j]!=-1)
                    printf("%d ", frames[j]);
                else printf("- ");
            }
            printf("\n");
            continue;
        }
        int pos=-1, farthest=i+1;
        for(int j=0; j<frameCount; j++){
            int k;
            for(k=i+1; k<n; k++){
                if(frames[j]==pages[k]){
                    if(k>farthest){
                        farthest=k;
                        pos=j;
                    }
                    break;
                }
            }
            if(k==n){
                pos=j;
                break;
            }
        }
        if(pos==-1) pos=0;
        frames[pos]=pages[i];
        pageFaults++;
        printf("Page %d -> ", pages[i]);
        for(int j=0; j<frameCount; j++){
            if(frames[j]!=-1) printf("%d ", frames[j]);
            else printf("- ");
        }
        printf("\n");
    }
    printf("Total page faults = %d\n", pageFaults);
}
int main(){
    int pages[50], n, frameCount, choice
```

O/p:

```
enter Numbers of pages : 5
enter page reference string: 7 7 4 0 1
enter number of frames : 3
Choose A
--- FIFO ---
page 7 -> 7 - -
page 7 -> 7 - -
page 4 -> 7 4 -
page 0 -> 7 4 0
page 1 -> 1 4 0
page faults = 4.
--- LRU ---
page 7 -> 7 - -
page 7 -> 7 - -
page 4 -> 7 4 -
page 0 -> 7 4 0
page 1 -> 1 4 0
page faults = 4
--- optimal ---
page 7 -> 7 - -
page 7 -> 7 - -
page 4 -> 4 - -
page 0 -> 0 - -
page 1 -> 1 - -
```